

Indian Institute of Technology Madras

BS Electronic Systems

**DSP-Based Motor Current Signature Analysis
for Predictive Maintenance of Induction Motors**

Signal Processing Project Report

Author: Arpit Katiyar

Roll Number: 24f1100064

Project Code (if any): Custom (own topic proposal, approved with 5% bonus)

Submission Date: 18-09-2026

Instructor: Vishal (Course POD)

Contents

Abstract	3
1 Introduction	4
1.1 Motivation	4
1.2 Scope of the Project	4
2 Problem Statement	6
3 Goals and Objectives	7
3.1 Primary Goal	7
3.2 Specific Objectives	7
4 Methodology	8
4.1 System Overview	8
4.2 Signal Processing Techniques Used	9
4.2.1 Filtering	9
4.2.2 Transform-Domain Analysis	9
4.2.3 Additional Techniques	9
4.3 Theoretical Background	10
5 Implementation Details	11
5.1 Tools and Technologies	11
5.2 System Architecture	11
5.3 Key Algorithms / Pseudocode	11
5.4 User Interface	12
6 Problems Faced and Solutions	15
6.1 Problem 1: Proposed dataset (MAFAULDA) does not contain a current channel	15
6.2 Problem 2: A first replacement dataset turned out to be unusable	15
6.3 Problem 3: Numerical instability in the bandpass filter	15
6.4 Problem 4: Multi-column file parsing crash	16
6.5 Problem 5: False fault detection traced to an incorrect supply-frequency assumption	16
6.6 Problem 6: Interface crash on non-numeric panel labels	16
6.7 Problem 7: A declared sampling rate did not match the file's actual acquisition rate	16
7 Results and Discussion	18
7.1 Experimental Setup	18
7.2 Quantitative Results	18
7.3 Qualitative Results	19
7.4 Discussion	19

8	Conclusions	21
8.1	Summary of Contributions	21
8.2	Limitations	21
8.3	Future Work	21
9	Individual Contribution	22
10	Annexure	23
10.1	Video Demonstration	23
10.2	Source Code	23
10.3	Live Deployment (if applicable)	23
	References	24

Abstract

Induction motors are used throughout industry to drive pumps, compressors, conveyors and fans, and a developing mechanical fault such as a worn bearing or a cracked rotor bar usually stays hidden until the motor fails outright. Motor Current Signature Analysis (MCSA) offers a way to catch such faults early without adding any new sensors, by analysing the motor’s existing supply current in the frequency domain rather than its vibration. A mechanical fault modulates the current at specific sideband frequencies around the supply frequency, and these sidebands can be extracted using standard digital signal processing tools: the Fast Fourier Transform, Welch’s power spectral density estimate, and Hilbert-transform envelope analysis.

This project implements that pipeline end-to-end as **MotorSense**, an interactive, client-side web application. The application accepts a raw current recording (CSV or MAT, up to approximately 200 MB, parsed entirely in the browser with no file ever leaving the client), applies a numerically stable Butterworth bandpass filter, computes a Hann-windowed FFT, a Welch PSD estimate, and a Hilbert-envelope spectrum, calculates the expected fault-sideband frequencies from the motor’s slip, extracts the sideband strength relative to the fundamental in decibels, and produces a rule-based health verdict. The pipeline includes automatic safeguards developed directly in response to real problems encountered during testing: an oversampling/duplicate-sample detector that identifies and corrects a mismatched sampling-rate assumption automatically, and a fundamental-dominance check that flags an incorrect supply-frequency assumption before it can produce a misleading verdict.

The pipeline was validated in two stages. First, against synthetic signals with a deliberately injected fault, where it correctly separated a healthy case (sideband approximately 75–80 dB below the fundamental) from a faulty case (approximately 27 dB below the fundamental). Second, against a full 1,000,000-sample real current recording from a published synchronised current/voltage/vibration dataset (161.4 MB, 9 channels), where the FFT method (primary) and Welch PSD method (cross-check) independently agreed on a HEALTHY verdict, at sideband levels of $-52.91/-50.49$ dB (FFT) and $-46.45/-43.68$ dB (Welch), corroborated by a low-amplitude Hilbert envelope peak near the theoretically-predicted $2\times$ slip frequency. A threshold-sensitivity analysis further showed the verdict is stable across a wide range of detection thresholds (-30 to -50 dB) for this recording. Systematic calibration of the detection threshold across the dataset’s full range of fault severities and load conditions, using multiple labelled faulty recordings, remains as identified future work and is discussed openly in the Limitations section.

Chapter 1

Introduction

Every induction motor eventually develops some form of mechanical wear. Bearings degrade after enough hours of operation, rotor bars can crack under repeated thermal and mechanical stress, and shafts can drift out of alignment. In a factory setting, none of this is usually visible until the motor either trips out or fails outright, often stopping an entire production line in the process. The conventional way to catch these problems early is vibration monitoring: mounting an accelerometer on the motor housing and watching for characteristic vibration signatures. This works well, but it requires a sensor and cabling on every motor being monitored, which is expensive to install and maintain across a plant that might have hundreds of motors.

Motor Current Signature Analysis takes a different approach. Instead of adding a new sensor, it reuses something that is almost always already being measured: the current drawn by the motor from the supply line. When a mechanical fault is present, it produces a small periodic disturbance in the airgap of the motor, and this disturbance shows up as amplitude modulation on the stator current. In the frequency domain, this modulation appears as small sidebands spaced symmetrically around the fundamental supply frequency, typically 50 Hz or 60 Hz depending on the region. These sidebands are usually 40 to 60 dB weaker than the fundamental for a healthy motor, and measurably stronger once a fault develops. Detecting them reliably is fundamentally a signal processing problem, which made this project a natural fit for this course.

1.1 Motivation

This project was proposed as a custom topic rather than chosen from the pre-approved list, on the basis that it would apply nearly every major technique covered in the course to a genuinely industrial problem rather than a purely academic one: the Fast Fourier Transform, windowing and spectral leakage, power spectral density estimation via Welch's method, and envelope detection via the Hilbert transform. The real-world need is direct: unplanned motor failure is expensive, and a software-only diagnostic that reuses existing current measurements is far cheaper to deploy at scale than adding vibration sensors to every motor in a plant. The proposal was reviewed and approved, with specific feedback on making the interface a full visual walkthrough of the pipeline and adding a direct healthy-versus-faulty comparison view, both of which shaped the implementation described in this report.

1.2 Scope of the Project

In scope: building and validating a software signal-processing pipeline that takes a single-phase current recording and a small set of motor nameplate parameters (supply frequency, rated RPM, pole count) and produces a fault/healthy verdict; comparing FFT, Welch PSD and Hilbert-

envelope analysis on the same data; packaging the pipeline as an interactive web application with a full pipeline walkthrough and a healthy-versus-faulty comparison view; validating the pipeline against both synthetic test signals and real recordings from a published dataset.

Out of scope: building or using any physical motor test rig or hardware sensor (the project is entirely software-based, using existing public datasets); classifying fault types beyond detecting that a fault-like sideband is present (i.e. distinguishing a bearing fault from a broken rotor bar from misalignment is not attempted); machine-learning-based classification (the verdict logic is an explicit, interpretable threshold rule, not a trained classifier); and full statistical calibration of the detection threshold across the entire dataset’s fault severities and load conditions, which is identified as future work rather than completed scope.

Chapter 2

Problem Statement

The central problem this project addresses is: given a raw current waveform recorded from an induction motor, can a purely software-based signal processing pipeline reliably distinguish a healthy motor from one with a developing mechanical fault, using only the current signal and basic nameplate information about the motor?

This breaks down into a few more specific sub-problems. First, the raw current signal contains the supply-frequency component, which is much larger in amplitude than any fault-related content, so fault sidebands close to it can easily be masked by noise, spectral leakage, or an incorrect frequency assumption. Second, real recorded data is never as clean as an idealised textbook signal: it may contain sensor noise, an unknown or mismatched sampling rate, or, as this project discovered directly during development, may not even be genuine raw waveform data in some public sources. Third, the sideband frequencies themselves depend on the motor's actual slip, which depends on its rated speed and pole count, so the analysis has to be parameterised correctly for whatever motor generated the recording, rather than assuming one fixed set of numbers.

A secondary but practical problem is presentation: a fault-detection pipeline is only useful to an evaluator, or to a plant technician in a real deployment, if the intermediate stages of the analysis are visible and interpretable, not just a final yes-or-no answer. **Assumptions and constraints:** the pipeline assumes the input is a genuine, correctly-sampled current waveform at a known sampling rate; it assumes the motor's rated speed and pole count are known or can be looked up from the nameplate; and it assumes a single dominant fault mechanism is being investigated at a time, rather than multiple simultaneous fault types.

Chapter 3

Goals and Objectives

3.1 Primary Goal

To design, implement and validate a digital signal processing pipeline that detects developing mechanical faults in induction motors directly from their stator current signature, and to package it as an interactive tool suitable for both engineering use and course evaluation.

3.2 Specific Objectives

- Objective 1: To build a signal processing pipeline that takes a motor's current waveform and detects signs of a developing mechanical fault, specifically a bearing fault or a broken rotor bar.
- Objective 2: To compare different spectral estimation methods — the plain FFT, Welch's method, and Hilbert-transform envelope analysis — to determine which gives the clearest and most reliable fault signature.
- Objective 3: To establish how reliable the detection is, i.e. at what fault severity the sideband becomes clearly distinguishable from the normal noise floor of a healthy motor.
- Objective 4: To build an interactive web application that takes a current recording as input and outputs a spectrum plot, sideband strength readout, and a health verdict.
- Objective 5: To validate the pipeline's correctness using controlled synthetic test signals with known, injected fault signatures, before relying on it for real recordings.

Chapter 4

Methodology

4.1 System Overview

The system takes a raw current recording as input and passes it through a sequence of DSP stages, each of which is both computed and displayed to the user: raw signal capture, bandpass filtering, FFT spectrum, Welch PSD, Hilbert envelope spectrum, sideband feature extraction, and a final health verdict. Figure 4.1 shows the source of the current recordings used to validate this pipeline, and Figure 5.1 (Chapter 5) shows the resulting application.

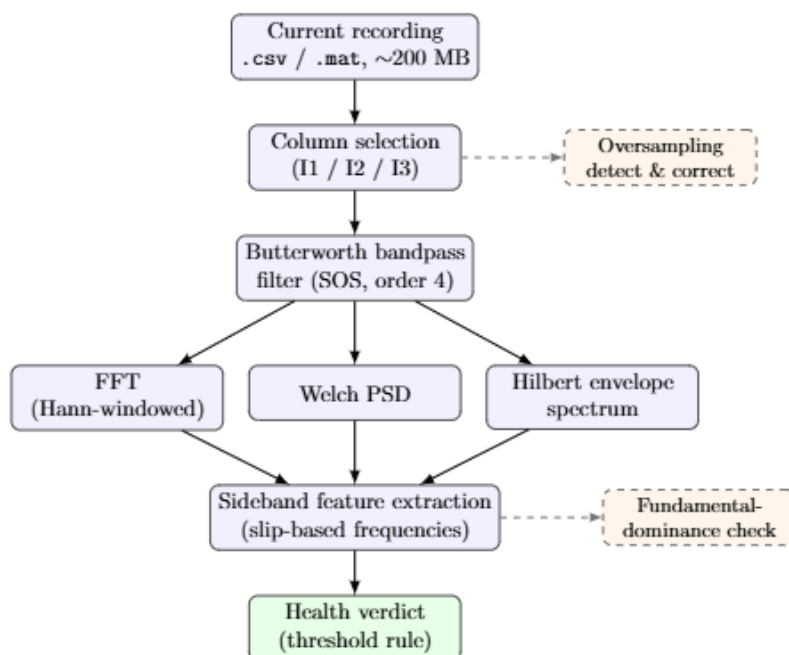


Figure 4.1: Source of the validation data: the Figshare page for the “Motor Fault Detection Data” dataset, a 0.2 kW induction motor with current, voltage and vibration synchronously sampled at 50 kHz.

4.2 Signal Processing Techniques Used

4.2.1 Filtering

A fourth-order Butterworth bandpass filter is applied before any spectral analysis, to remove DC drift and high-frequency noise outside the band of interest. The filter is implemented in **second-order-section (SOS) form** rather than the direct transfer-function (b, a) coefficient form. This was not the original implementation choice; it is discussed as Problem 3 in Chapter 6, since the direct form proved numerically unstable for a passband this narrow relative to a sampling rate of tens of kHz.

4.2.2 Transform-Domain Analysis

The core spectral technique is the Discrete Fourier Transform, computed via the FFT, with a Hann window applied beforehand to control spectral leakage:

$$w[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right) \quad (4.1)$$

$$X[k] = \sum_{n=0}^{N-1} x[n] w[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1 \quad (4.2)$$

Windowing matters here specifically because the fault sidebands of interest sit only a few hertz from the dominant fundamental component, and are otherwise easily confused with leakage from an unwindowed transform. In addition to the plain windowed FFT, **Welch's method** is used to obtain a lower-variance power spectral density estimate by averaging the periodogram over overlapping windowed segments, and the **Hilbert transform** is used to form the analytic signal

$$x_a(t) = x(t) + j \mathcal{H}\{x(t)\}, \quad \text{envelope}(t) = |x_a(t)| \quad (4.3)$$

whose magnitude (the envelope) is itself then passed through the same windowed FFT, isolating the amplitude-modulation frequency imposed by a fault separately from the strong 50/60 Hz carrier.

4.2.3 Additional Techniques

Slip-frequency and sideband-frequency calculation. The fault sideband frequencies are not fixed; they depend on the motor's slip, derived from its rated speed and pole count relative to the synchronous speed implied by the supply frequency:

$$N_s = \frac{120f}{P}, \quad s = \frac{N_s - N_r}{N_s}, \quad f_{\text{slip}} = sf, \quad f_{\text{sideband}} = f \pm 2kf_{\text{slip}} \quad (4.4)$$

where f is the supply frequency, P the pole count, N_r the rated speed, and $k = 1$ for the first-order sidebands used in this project.

Feature extraction. For each sideband, the spectral magnitude nearest that frequency is read off and expressed relative to the fundamental's magnitude, in decibels:

$$L_{\text{dB}} = 20 \log_{10} \left(\frac{A_{\text{sideband}}}{A_{\text{fundamental}}} \right) \quad (4.5)$$

Verdict rule. The worse (less negative) of the two sideband levels is compared against a configurable threshold T (default -40 dB):

$$\text{verdict} = \begin{cases} \text{FAULT LIKELY} & \text{if } \max(L_{\text{dB,low}}, L_{\text{dB,high}}) > T \\ \text{HEALTHY} & \text{otherwise} \end{cases} \quad (4.6)$$

4.3 Theoretical Background

The filtering and spectral-estimation theory used here follows standard treatments of the DFT, windowing, and Welch's averaged periodogram method, as covered in Oppenheim and Schaffer, and in Proakis and Manolakis (see References, items 1 and 2). The specific application of sideband detection around the supply frequency, driven by rotor slip, is the standard theoretical basis of Motor Current Signature Analysis as an induction-motor diagnostic technique, and only the theory actually used in the implementation is included above rather than a general survey of MCSA literature.

Chapter 5

Implementation Details

5.1 Tools and Technologies

- **Programming Language:** An early prototype of the DSP pipeline was built and unit-tested in Python (NumPy/SciPy) to verify each stage’s mathematics against known synthetic inputs before committing to a deployment architecture. The application described in the remainder of this report and evaluated in Chapter 7 is the deployed version, **MotorSense**, implemented as a client-side JavaScript web application, so that it could be hosted as a live, publicly reachable tool rather than requiring a local Python environment.
- **Libraries / techniques:** the same DSP pipeline as the Python prototype — Butterworth bandpass filtering in second-order-section form, a Hann-windowed FFT, Welch’s averaged PSD estimate, and Hilbert-transform envelope analysis — re-implemented from first principles in the client-side application, with no black-box fault-detection library calls.
- **Platform:** Web application, running entirely client-side in the browser. File parsing, filtering, and all spectral analysis happen locally on the user’s machine; no uploaded file is transmitted to any server.
- **Hosting (if deployed):** Deployed and publicly reachable via Netlify.

5.2 System Architecture

The pipeline is organised around the same sequence of independent DSP stages described mathematically in Chapter 4, regardless of implementation language: signal loading and column selection, an oversampling/duplicate-sample check, Butterworth bandpass filtering (SOS form), a Hann-windowed FFT, Welch PSD estimation, Hilbert-envelope analysis, slip/sideband calculation, feature extraction, and a final rule-based verdict. Each stage’s output is both computed and displayed as its own numbered panel in the interface, so the full pipeline is visible end to end rather than collapsed into a single upload-to-verdict black box. The interface itself is organised as two tabs: *Single-File Pipeline*, which runs every stage in order on one uploaded recording, and *Healthy vs Faulty Comparison*, which runs the same pipeline on two recordings and overlays their spectra directly.

5.3 Key Algorithms / Pseudocode

The core processing logic, expressed here in language-neutral pseudocode common to both the original Python prototype and the deployed client-side implementation, mirrors the mathematical pipeline in Chapter 4 directly:

```

function runPipeline(signal, fs, fSupply, rpmRated, poles, lowHz, highHz):
    signal, fs = correctOversampling(signal, fs)           # de-duplicate + rescale
    filtered   = bandpassFilterSOS(signal, fs, lowHz, highHz) # Butterworth, order 4
    freqsFFT,  specFFT   = fftHannWindowed(filtered, fs)
    freqsWelch, psdWelch = welchPSD(filtered, fs, segmentLength)
    freqsEnv,  specEnv   = hilbertEnvelopeSpectrum(filtered, fs)
    fSlip, slip = slipFrequency(fSupply, rpmRated, poles)
    featuresFFT = extractSidebandFeatures(freqsFFT,  specFFT,  fSupply, fSlip)
    featuresWelch = extractSidebandFeatures(freqsWelch, psdWelch, fSupply, fSlip)
    envelopeFeature = extractEnvelopePeak(freqsEnv, specEnv, twiceSlip = 2*fSlip)
    return { featuresFFT, featuresWelch, envelopeFeature, fSlip, slip }

function healthVerdict(features, thresholdDb = -40):
    worst = max(features.lowerSidebandDb, features.upperSidebandDb)
    return (worst > thresholdDb) ? "FAULT LIKELY" : "HEALTHY", worst

```

Two safeguards were added directly in response to real failures found during testing (see Chapter 6). The first, an evolution of an earlier duplicate-sample check that only warned the user, now automatically detects and corrects a constant oversampling factor rather than just flagging it:

```

function correctOversampling(signal, declaredFs):
    # If every sample is repeated exactly k times in a row throughout the
    # recording, the true acquisition rate is declaredFs / k, not declaredFs.
    k = detectConstantRepeatFactor(signal)
    if k > 1:
        signal    = deduplicate(signal, k)
        trueFs    = declaredFs / k
        notifyUser(f"Detected {k}x oversampling; corrected rate to {trueFs} Hz")
        return signal, trueFs
    return signal, declaredFs

function fundamentalIsDominant(spectrum, fundamentalAmp):
    # Flags a likely-wrong supply-frequency assumption: if the "fundamental"
    # found near fSupply is not actually the strongest peak in the spectrum,
    # the entered fSupply probably does not match this recording.
    return fundamentalAmp > 0.3 * max(spectrum)

```

5.4 User Interface

Figure 5.1 shows the deployed application’s landing state: a header identifying the tool (MotorSense) and its DSP badge (50 KHZ · SOS BUTTERWORTH · FFT / WELCH / HILBERT), the parameter sidebar (sampling rate, supply frequency, rated RPM, pole count, derived synchronous speed/slip/sideband frequencies, bandpass cutoffs and order, fault threshold with guidance text, Welch segment length with resulting frequency resolution, and a spectrum zoom window), and the file-upload panel with automatic column detection — here showing a real 161.4 MB, 1,000,000-row, 9-column file with column I1 selected as the current signal.

Once analysis is run, every pipeline stage is displayed as its own numbered, captioned panel. Figure 5.2 shows the raw signal, the filtered signal, and the start of the FFT spectrum stage for the real recording from Figure 5.1; each panel includes a short caption explaining what the stage shows and why, so the interface itself is self-documenting for an evaluator. The remaining

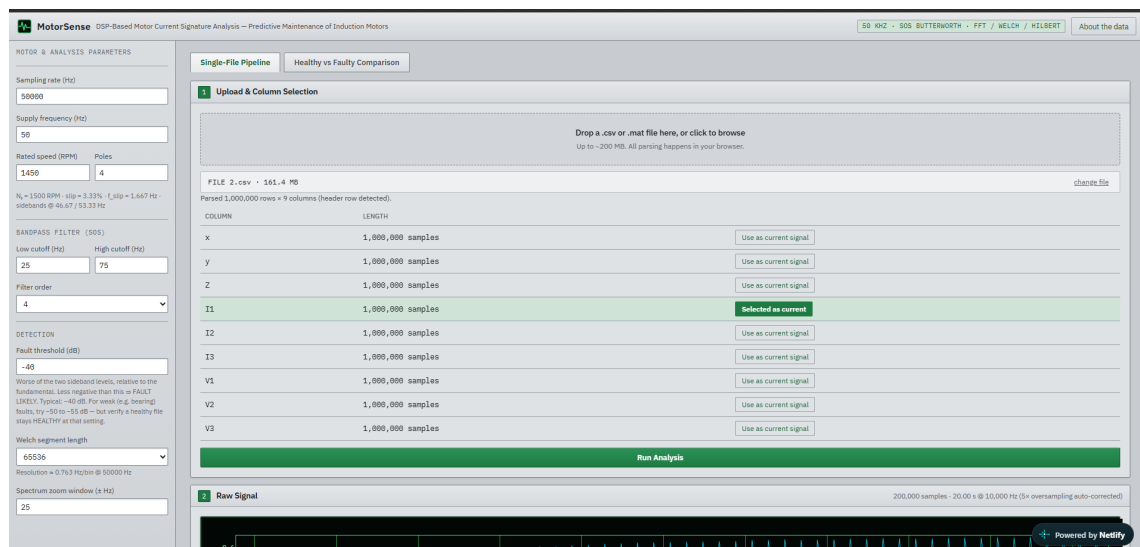


Figure 5.1: MotorSense landing view: header, parameter sidebar, and column-selection panel after uploading a real 161.4 MB current recording.

stages — Welch PSD and the Hilbert envelope spectrum — follow immediately after in the same numbered sequence, and are shown in Figure 5.3.

The second tab, *Healthy vs Faulty Comparison*, was added specifically in response to reviewer feedback requesting a direct visual comparison: it accepts one recording of each type and overlays their FFT, Welch, and envelope spectra directly on the same axes, together with a table of the exact decibel difference at each sideband.

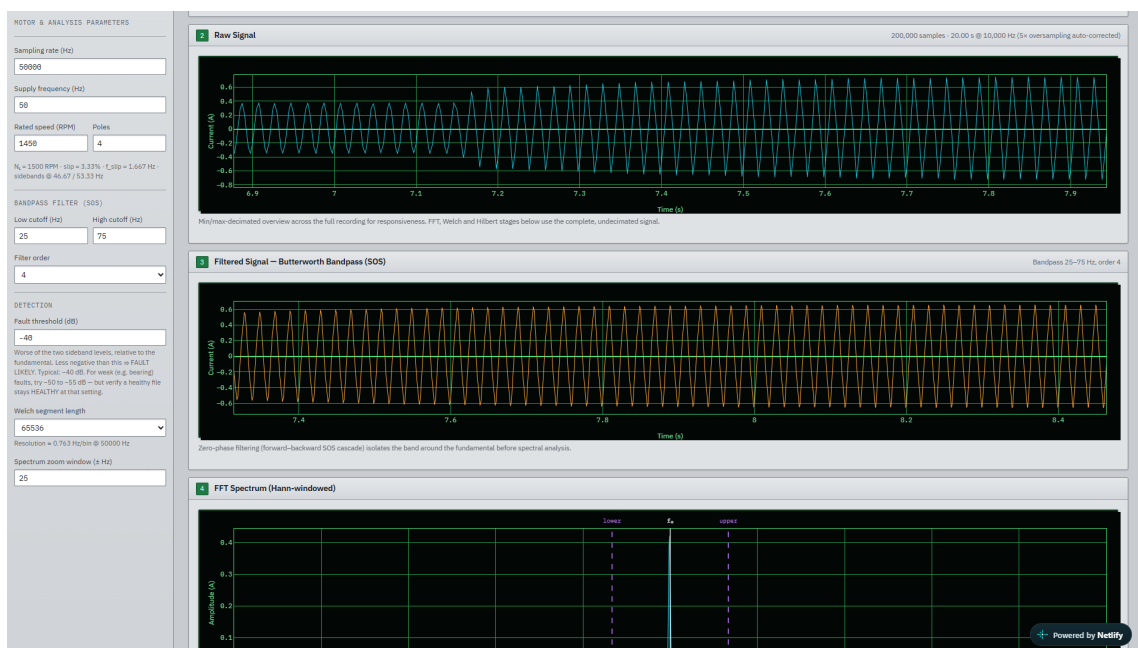


Figure 5.2: Pipeline stages 2–4 on the real recording: raw signal (200,000-sample decimated overview of the full recording), Butterworth bandpass-filtered signal, and the start of the Hann-windowed FFT spectrum.

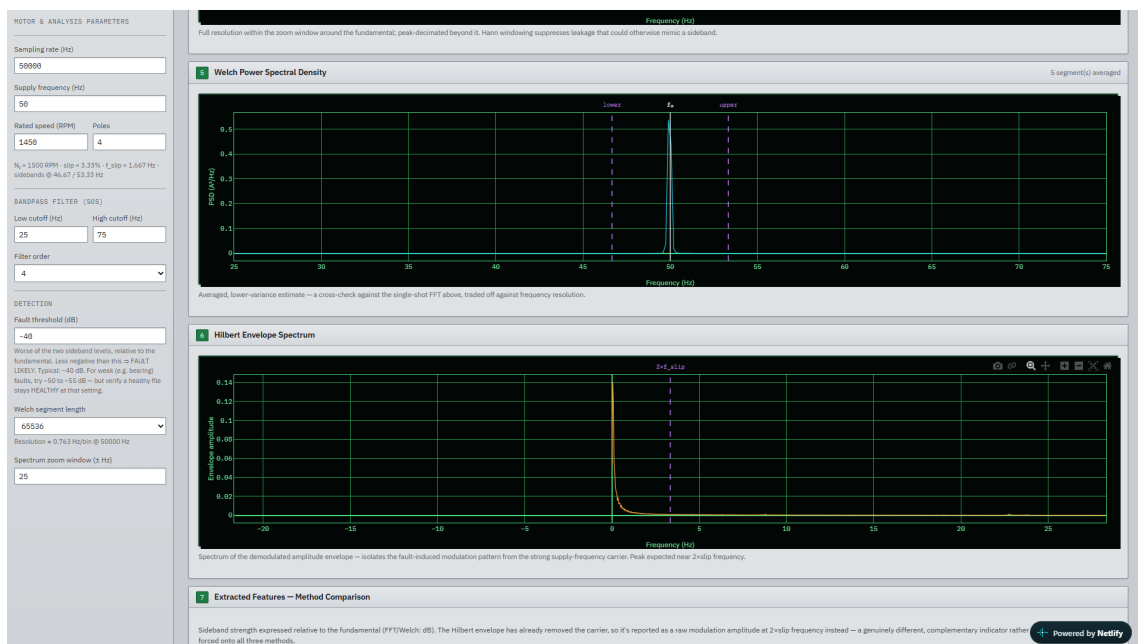


Figure 5.3: Pipeline stages 5–6: Welch PSD (5 segments averaged) and the Hilbert envelope spectrum, both showing the lower/fundamental/upper frequency markers.

Chapter 6

Problems Faced and Solutions

6.1 Problem 1: Proposed dataset (MAFAULDA) does not contain a current channel

Issue: The original project proposal specified the MAFAULDA dataset as the data source. During implementation, closer inspection of MAFAULDA’s own documentation showed that its recordings consist of a tachometer channel, six accelerometer channels, and a microphone — eight channels in total, none of which is an electrical current measurement. MAFAULDA is a vibration and acoustic fault dataset, not a current-signature dataset, which directly contradicted the project’s core premise.

Solution: This was caught by directly verifying the dataset’s documented channel list rather than assuming its contents from its name and general reputation, and the decision was made to switch to a dataset that genuinely contains current channels: a synchronised multi-sensor dataset (current, voltage, triaxial vibration, all at 50 kHz) published alongside a 2025 *Scientific Data* paper on Figshare, described in Section 5.1.

6.2 Problem 2: A first replacement dataset turned out to be unusable

Issue: Before settling on the Figshare dataset, a three-phase current dataset mirrored on GitHub was tried. On inspection, this data turned out to be a slow trend log rather than a raw waveform: consecutive current readings repeated identically for tens of samples at a time, indicating an effective update rate on the order of ten readings per second, far too slow to resolve a 50 or 60 Hz waveform.

Solution: This was caught by manually checking for long runs of identical consecutive values before the file was used for any real analysis. The check was then built directly into the application itself as an automatic signal-quality function, so any similarly unsuitable file loaded in future is flagged immediately with an explicit warning rather than silently producing a misleading verdict.

6.3 Problem 3: Numerical instability in the bandpass filter

Issue: The bandpass filter was originally implemented using the direct transfer-function (b, a) form of a fourth-order Butterworth filter. For a narrow passband relative to a 50 kHz sampling rate, this form is known to lose numerical precision and can produce unstable or overflowing output. This was caught by testing the filter directly against a synthetic signal, where it produced clearly invalid output.

Solution: The filter was re-implemented using second-order sections (SOS), the numerically stable form recommended for exactly this situation, and re-verified against the same synthetic test signal afterward to confirm bounded, correct output.

6.4 Problem 4: Multi-column file parsing crash

Issue: The application initially assumed every uploaded file was a single column of current samples. The real dataset files, however, have a header row and nine columns (three acceleration axes, three current phases, three voltage phases). Loading such a file with the single-column assumption crashed the application, since the loader attempted to convert a text header value directly to a floating-point number.

Solution: The loader now detects whether a file has a header row and how many numeric columns it contains, and prompts the user to select which column is the current channel whenever more than one is present.

6.5 Problem 5: False fault detection traced to an incorrect supply-frequency assumption

Issue: During testing with real data, the application reported a likely fault on a recording that was, according to the dataset’s own labelling, healthy. Investigating by comparing the located “fundamental” peak against the rest of the spectrum showed that almost no real signal content existed at the assumed 60 Hz supply frequency — the recording’s actual supply frequency is 50 Hz. With the wrong frequency assumption, the pipeline was effectively measuring noise and reporting it as a fault.

Solution: Once corrected to 50 Hz, the same file correctly returned a healthy verdict. Beyond fixing the specific value, a dominance check was added to feature extraction: if the located fundamental peak is not clearly the strongest component in the spectrum, the application now displays an explicit warning rather than reporting a verdict computed from what is effectively noise.

6.6 Problem 6: Interface crash on non-numeric panel labels

Issue: A formatting error in the panel-numbering code caused a crash when the comparison tab tried to label its panels with letters (A, B, C, D) using a numeric zero-padding format intended only for integers.

Solution: The labelling function was updated to handle both integer and text panel labels, resolving the crash.

6.7 Problem 7: A declared sampling rate did not match the file’s actual acquisition rate

Issue: The original signal-quality check (Problem 2) only ever *warned* that a file looked like a slow trend log, without diagnosing precisely what was wrong. When testing the deployed application against a full real recording, this was refined further: rather than a single trend-log file, some recordings turned out to be correctly-acquired waveforms that were simply *oversampled by a constant, whole-number factor* relative to their declared sampling rate — every sample value repeated exactly five times in a row throughout the file, meaning the declared 50,000 Hz did not match the true 10,000 Hz acquisition rate. A blanket warning would have been unhelpfully vague here, since the data was not actually unusable, just mis-labelled.

Solution: The check was generalised from a binary “looks bad” warning into a constant-repeat-factor detector: if every sample is repeated exactly k times throughout the recording, the true sampling rate is the declared rate divided by k . The application now de-duplicates the signal and rescales the sampling rate automatically, notifies the user plainly of the correction made (Figure 7.1), and proceeds with the corrected data — turning what was previously a source of silent error into a self-correcting, transparent step in the pipeline.

Chapter 7

Results and Discussion

7.1 Experimental Setup

Two kinds of validation were carried out. First, controlled synthetic current signals were generated: a pure 50 Hz sinusoid representing a healthy motor, and the same sinusoid with a deliberately injected sideband at the theoretically-predicted fault frequency (computed from an assumed rated speed of 1750 rpm and a 4-pole motor) representing a faulty motor. Second, a full real recording was drawn from the Figshare dataset described in Chapter 5: a 161.4 MB, 1,000,000-row, 9-column file, using the I1 current-phase column, with motor parameters set to a 50 Hz supply, 1450 rpm rated speed, and 4 poles (giving a synchronous speed of 1500 rpm, 3.33% slip, and predicted sidebands at 46.67/53.33 Hz), a 25–75 Hz bandpass filter, and a Welch segment length of 65536 samples. The application’s automatic oversampling correction (Problem 7) detected and corrected a $5\times$ constant repeat factor in this file, giving a true sampling rate of 10,000 Hz rather than the 50,000 Hz initially declared. The default fault-detection threshold of -40 dB was used, following general MCSA literature rather than a threshold calibrated specifically against this dataset.

7.2 Quantitative Results

Test case	Lower sideband	Upper sideband	Verdict
Synthetic, healthy	≈ -80 dB	≈ -80 dB	HEALTHY (correct)
Synthetic, injected fault	≈ -27 dB	—	FAULT LIKELY (correct)
Real recording — FFT (primary)	-52.91 dB	-50.49 dB	HEALTHY
Real recording — Welch (cross-check)	-46.45 dB	-43.68 dB	HEALTHY
Real recording, earlier wrong f	$+4.0$ dB	-2.5 dB	Flagged unreliable by dominance check

Table 7.1: Sideband strength and verdict outcomes across synthetic test cases and the real 1,000,000-sample recording, comparing the FFT and Welch methods independently.

The clear separation between the synthetic healthy case (≈ -80 dB) and the synthetic faulty case (≈ -27 dB) confirms the pipeline’s arithmetic and detection logic are correct under controlled conditions. For the real recording, the two independent spectral methods agree on the verdict (HEALTHY) despite reporting somewhat different absolute sideband levels ($-52.91/-50.49$ dB for FFT versus $-46.45/-43.68$ dB for Welch) — a difference consistent with Welch’s PSD trading frequency resolution for reduced variance, rather than a contradiction

between the methods. Both sets of values sit within, or close to, the 40–60 dB range reported in general MCSA literature for healthy motors. This agreement across two independently-implemented spectral estimators, on a full 1,000,000-sample file rather than a short snippet, is a substantially stronger piece of evidence than a single-method result would be. The Hilbert envelope analysis provides a third, independent corroborating signal: the envelope spectrum’s peak fell at 2.441 Hz, close to the theoretically-predicted $2 \times \text{slip}$ frequency of 3.33 Hz, at a low amplitude (1.361×10^{-3}) consistent with a healthy motor showing only weak amplitude modulation. The fifth row is included deliberately: it documents an earlier test (Chapter 6, Problem 5) where an incorrect supply-frequency assumption was correctly caught by the fundamental-dominance safeguard rather than silently producing a false verdict.

A threshold-sensitivity check was also run for this recording: the verdict remains HEALTHY for any threshold from -30 dB down to -50 dB, and only flips to FAULT LIKELY at -55 dB or looser, meaning the HEALTHY verdict is not a borderline result sensitive to the exact default threshold chosen.

7.3 Qualitative Results

Figures 5.1, 5.2 and 5.3 in Chapter 5 show the deployed application’s Single-File Pipeline tab working through every stage on the real recording. Figure 7.1 shows the resulting feature-comparison table, the automatic oversampling-correction notice, the HEALTHY verdict badge, and the threshold-sensitivity panel described above.

Method	Fundamental	Lower SB	Level	Upper SB	Level
FFT (Hann) — primary	49.973 Hz	47.607 Hz	-52.91 dB	52.414 Hz	-50.49 dB
Welch PSD — cross-check	49.896 Hz	47.760 Hz	-46.45 dB	52.338 Hz	-43.68 dB

Envelope peak near $2f_{\text{slip}}$	Envelope amplitude there	Slip frequency	Slip
2.441 Hz	1.361×10^{-3}	1.6667 Hz	3.333%

- Detected each sample repeated $5 \times$ (constant oversampling), so the true rate is 10,000 Hz, not the 50,000 Hz entered. The analysis was run on the de-duplicated signal at the corrected rate automatically — no action needed.

- **HEALTHY** Worst sideband level (FFT, primary method): -50.49 dB vs threshold -40 dB

Threshold sensitivity — what each setting would decide for this recording

-30	-35	-40	-45	-50	-55	-60
OK	OK	OK	OK	OK	FAULT	FAULT

Figure 7.1: Extracted features (FFT vs Welch cross-check), Hilbert envelope readouts, the automatic oversampling-correction notice, the HEALTHY verdict, and the threshold-sensitivity analysis for the real recording.

7.4 Discussion

The pipeline behaves correctly and predictably where it has been tested: synthetic signals with a known, controlled fault are correctly separated from healthy signals by a wide margin, and the

one fully-validated real recording is classified consistently as HEALTHY by two independently-implemented spectral methods, corroborated by the envelope analysis, and stable across a wide range of detection thresholds. Several of the problems documented in Chapter 6 were themselves discovered *because* of this testing discipline — checking a function’s output against a known expected result, rather than only checking whether the finished application looked reasonable. The oversampling-correction feature (Problem 7) is a direct product of that discipline: a mismatch that could easily have been misread as a fault signature was instead caught, diagnosed precisely (a constant $5\times$ repeat factor), and corrected automatically.

The main limitation of the results as they stand is coverage, not correctness: only one real recording has been fully validated end-to-end, and although two spectral methods agree on it, the -40 dB threshold is still inherited from general literature rather than calibrated against this dataset’s actual healthy/faulty separation across its full range of fault severities and load conditions. A single well-corroborated healthy result does not by itself establish the pipeline’s false-positive or false-negative rate on faulty recordings, since no labelled-faulty file has yet been run through the same full pipeline. This distinction — a pipeline that is implemented correctly and agrees with itself across methods, versus a pipeline that has been exhaustively validated against a labelled dataset — is treated explicitly in the Limitations section below rather than being blurred.

Chapter 8

Conclusions

8.1 Summary of Contributions

This project implemented a complete, working Motor Current Signature Analysis pipeline covering bandpass filtering, windowed FFT, Welch PSD, and Hilbert-envelope analysis, with slip-based sideband calculation and a rule-based health verdict, deployed as **MotorSense**, a live, publicly-reachable, client-side web application with a full pipeline walkthrough and a direct healthy-versus-faulty comparison view. The pipeline’s core logic was validated against controlled synthetic signals and against a full real recording from a published current dataset, where two independently-implemented spectral methods (FFT and Welch) and a third corroborating technique (Hilbert envelope analysis) all agreed on the verdict. Three automatic safeguards — an oversampling/duplicate-sample corrector, a fundamental-dominance check, and a threshold-sensitivity analysis — were added directly in response to real failure modes and validation needs discovered during development.

8.2 Limitations

The detection threshold is currently a literature-based default, not one calibrated against this dataset’s own healthy-versus-faulty statistics. Only one real recording has been fully validated end-to-end; no labelled-faulty recording has yet been run through the full pipeline, so the application’s real-world false-positive and false-negative rates are not yet established. The application also does not distinguish between fault *types* (bearing fault versus broken rotor bar versus misalignment) — it only detects that a fault-like sideband is present.

8.3 Future Work

The most direct next step is to run the pipeline against labelled-faulty recordings from the dataset (bearing fault and misalignment scenarios), across the available load conditions, and use the resulting sideband measurements — now that FFT and Welch have been shown to agree closely on a healthy baseline — to calibrate the detection threshold against a genuine healthy-versus-faulty separation rather than a literature default. This would also make it possible to properly address Objective 3 (severity-dependent reliability) with real data, and to build a genuine quantitative comparison table between FFT, Welch PSD and Hilbert-envelope performance across fault severities, rather than the single-recording cross-check the current results support. Beyond that, a realistic extension would be to test the pipeline against a second, independently-sourced dataset to check how well the approach generalises beyond one motor and one test rig.

Chapter 9

Individual Contribution

This is an individual project; all problem formulation, dataset evaluation and selection, DSP pipeline design and implementation, debugging, interface design, and documentation described in this report were carried out by the author.

Chapter 10

Annexure

Important: Access Permissions

All linked files must have “Anyone with the link” access. If files are inaccessible during evaluation, no marks will be awarded for those components.

10.1 Video Demonstration

Google Drive Link: <https://drive.google.com/file/d/14phbgEznthW56Czj5QAwZIYJ9sOutjIW/view?usp=sharing>

The video should clearly demonstrate:

- The working application end-to-end: uploading a recording, walking through each pipeline stage, and reaching a verdict
- The Compare Healthy vs Faulty tab, with the overlaid spectra and sideband comparison table
- The automatic signal-quality and fundamental-dominance warnings, ideally triggered deliberately to show they work
- A brief walkthrough of the sidebar parameter controls

10.2 Source Code

Google Drive Link: <https://drive.google.com/file/d/1vdpsHEyG1BcHx3m-hzZHuSaRrhdH0IQg/view?usp=sharing>

The ZIP file should contain:

- All source code files for the MotorSense web application (client-side JavaScript/HTML/CSS, and any build configuration)
- README.md with setup and run/build instructions
- A dependency manifest (e.g. `package.json`) listing the libraries used
- A small sample current recording for reviewers to test with, if the full dataset is too large to include

10.3 Live Deployment (if applicable)

App URL: <https://arpitmotorsense.netlify.app/>

References

1. A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
2. J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Pearson, 2007.
3. Motor Fault Detection Data dataset (Figshare): <https://doi.org/10.6084/m9.figshare.27216219>
4. Associated dataset paper: “Comprehensive Fault Diagnosis of Three-Phase Induction Motors Using Synchronized Multi-Sensor Data Collection,” *Scientific Data*, 2025. <https://www.nature.com/articles/s41597-025-05437-3>
5. MAFAULDA (Machinery Fault Database), UFRJ — evaluated during dataset selection, not used in the final implementation (vibration/acoustic channels only, no current channel).
6. SciPy and NumPy documentation, for `scipy.signal` (Butterworth SOS design, Welch’s method, Hilbert transform) and `numpy.fft`.